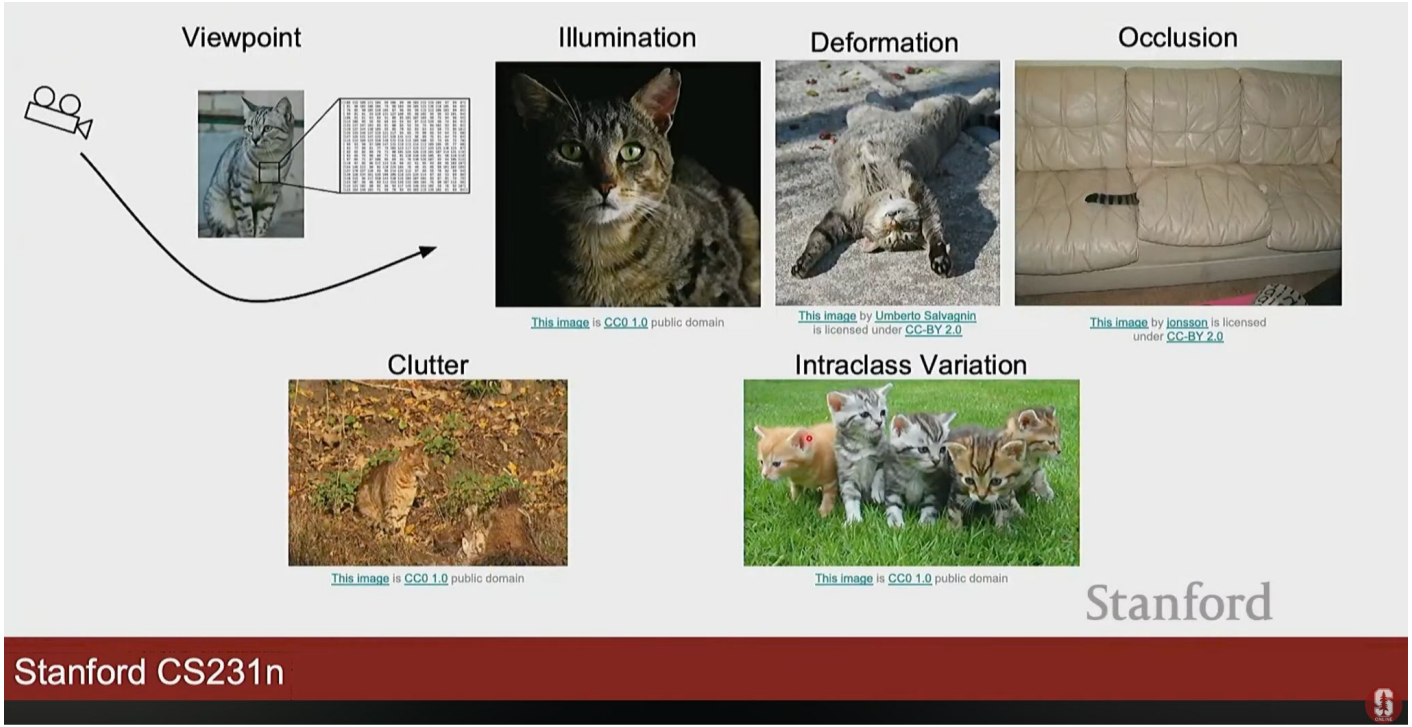
Newton School of Technology

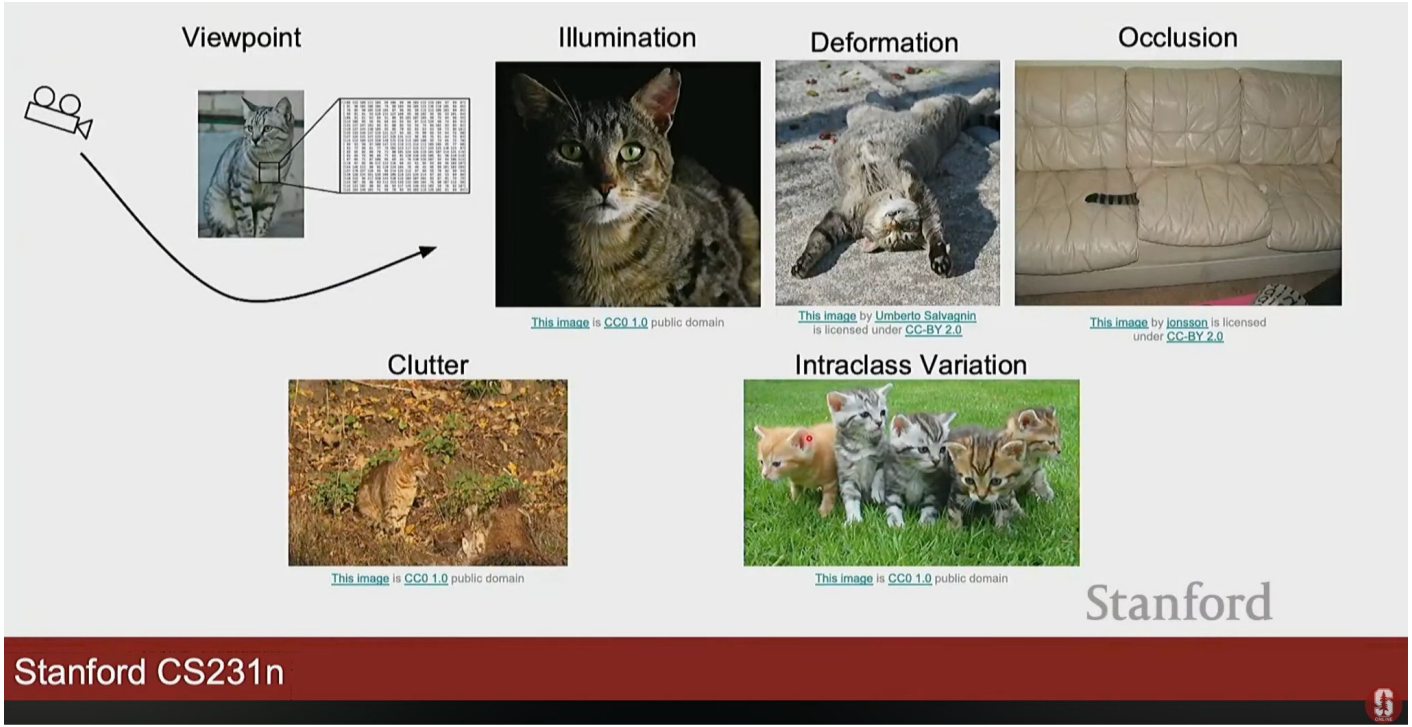
Join the lecture online on your dashboard.

Regularization In Deep Learning

In machine learning the data we work with has incredible variability



Teaching a model to handle all of this is hard



Let's have a look at some classic pitfalls

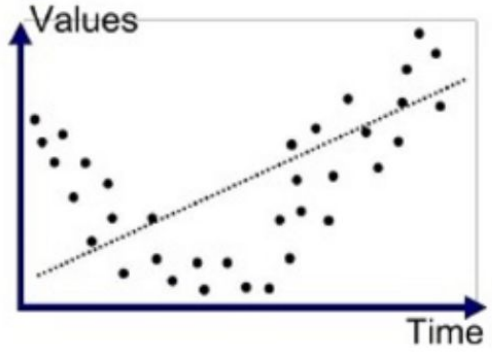
Issues in Model Fitting

Model Fitting

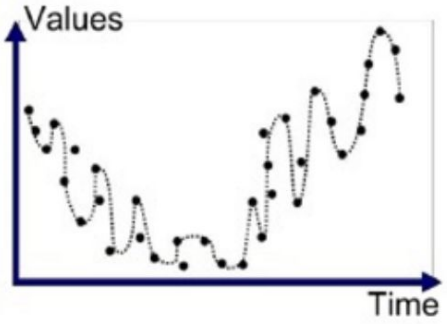
How well a model performs on training/evaluation datasets will define its characteristics

	Underfit	Overfit	Good Fit
Training Dataset	Poor	Very Good	Good
Evaluation Dataset	Very Poor	Poor	Good

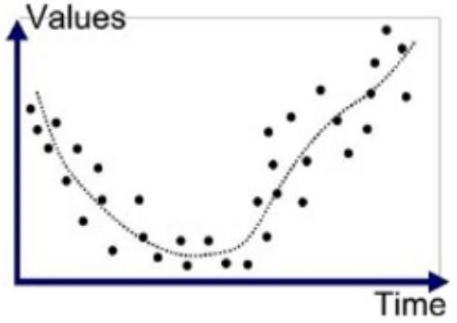
Model Fitting



Underfitted



Overfitted



Good Fit/Robust

Variations of model fitting

Bias Variance

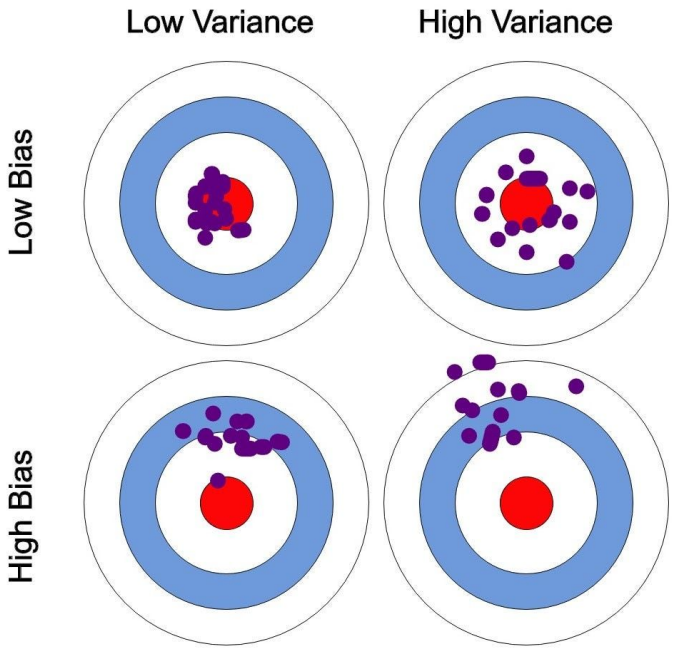
- Prediction errors [2]

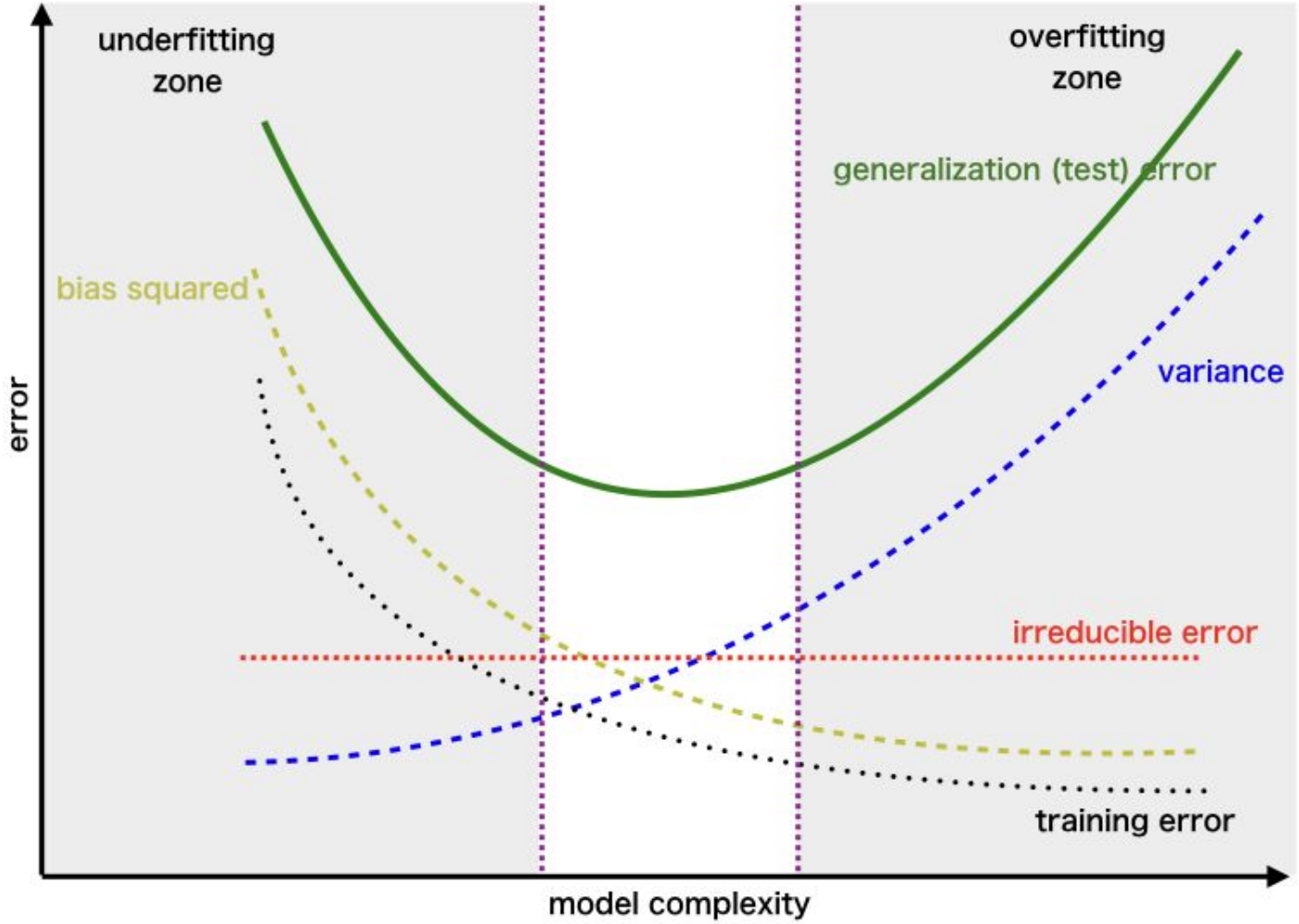
$$Error(x) = \underbrace{(E[\hat{f}(x)] - f(x))^2}_{(\text{Bias})^2} + \underbrace{E[\hat{f}(x) - E[\hat{f}(x)]]^2}_{\text{Variance}}$$

$$Error = (Avg(Predicted) - True)^2 + Avg(Predicted - Avg(Predicted))^2$$

Bias/Variance

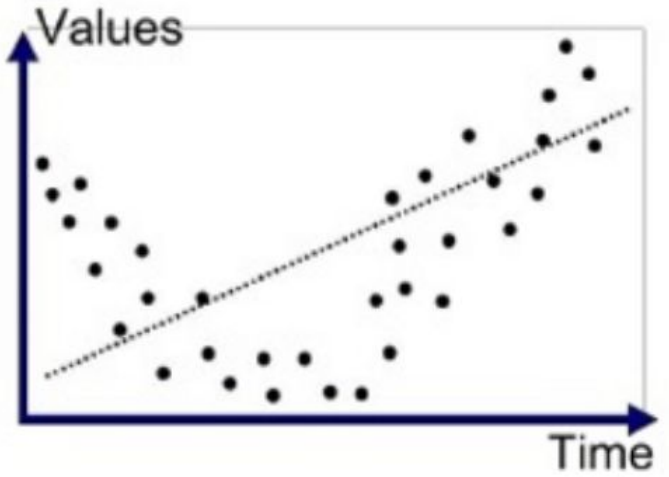
- **Bias**
  - Represents the extent to which average prediction over all datasets differs from the desired regression function
- **Variance**
  - Represent the extent to which the model is sensitive to the particular choice of data set




underfitting
zone
overfitting
zone
generalization (test) error
bias squared
variance
irreducible error
training error
error
model complexity

Let’s Counter Underfitting

- What causes underfitting?
  - Model capacity is too small to fit the training dataset as well as generalize to new dataset.
  - High bias, low variance



Underfitted

How do we fix this ?

Let’s Counter Underfitting

It’s so simple, just turn it into an overfit model!

Solution

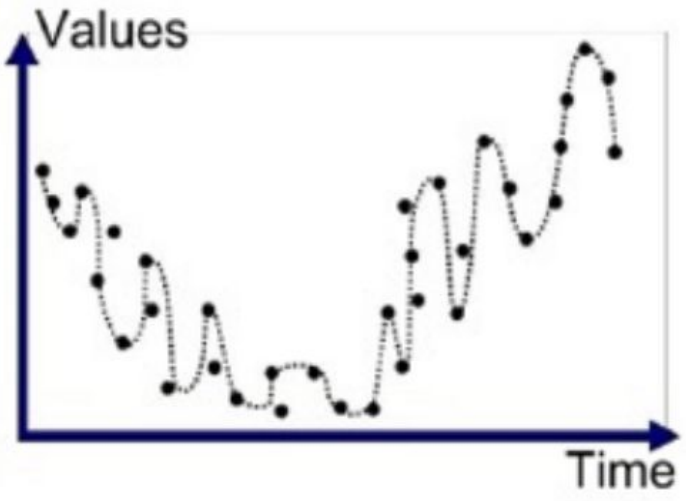
- Increase the capacity of the model
- Examples:
  - Increase number of layers, neurons in each layer, etc.

Result:

- Lower Bias
- *Underfit → Good Fit?*

Let’s Counter Overfitting

- What cause overfitting?
  - Model capacity is so big that it adapts too well to training samples, unable to generalize well to new, unseen samples.
  - Low bias, high variance.



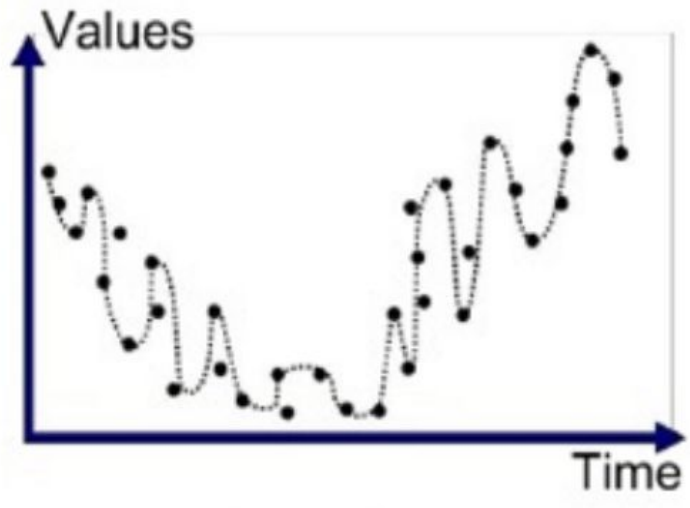
Overfitted

How do we fix this ?

Let’s Counter Overfitting

- Solution
  - **Regularization**
- **But How?**

By adding a penalty term to the cost function, which discourages overly complex models



Overfitted

Regularization Definition

- *Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error.* [4]

Generalization error (also known as **out-of-sample error** or **risk**) is a measure of how accurately an algorithm can predict outcome values for previously unseen data. It

Training error, or empirical risk, is the average loss or discrepancy calculated between predicted and actual target values on the same dataset used to train a machine learning model. It measures

Regularization Techniques

Summary: The Four Principles of Regularization

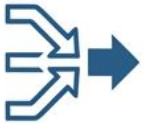
Make it Smoother

- L2 Regularization
- Early Stopping

More Data

- Data Augmentation
- Transfer Learning

Combine Models



- Ensembling
- Dropout

Wider Minima

- Weight Noise
- SGD Implicit Regularization

It is actually much more complex

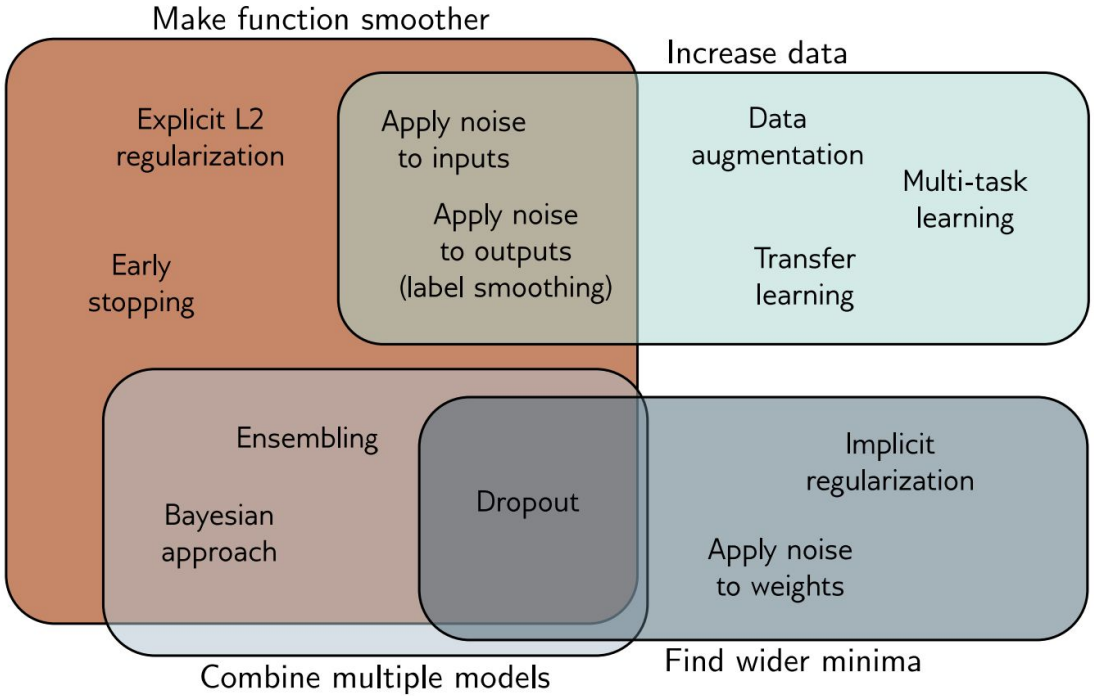
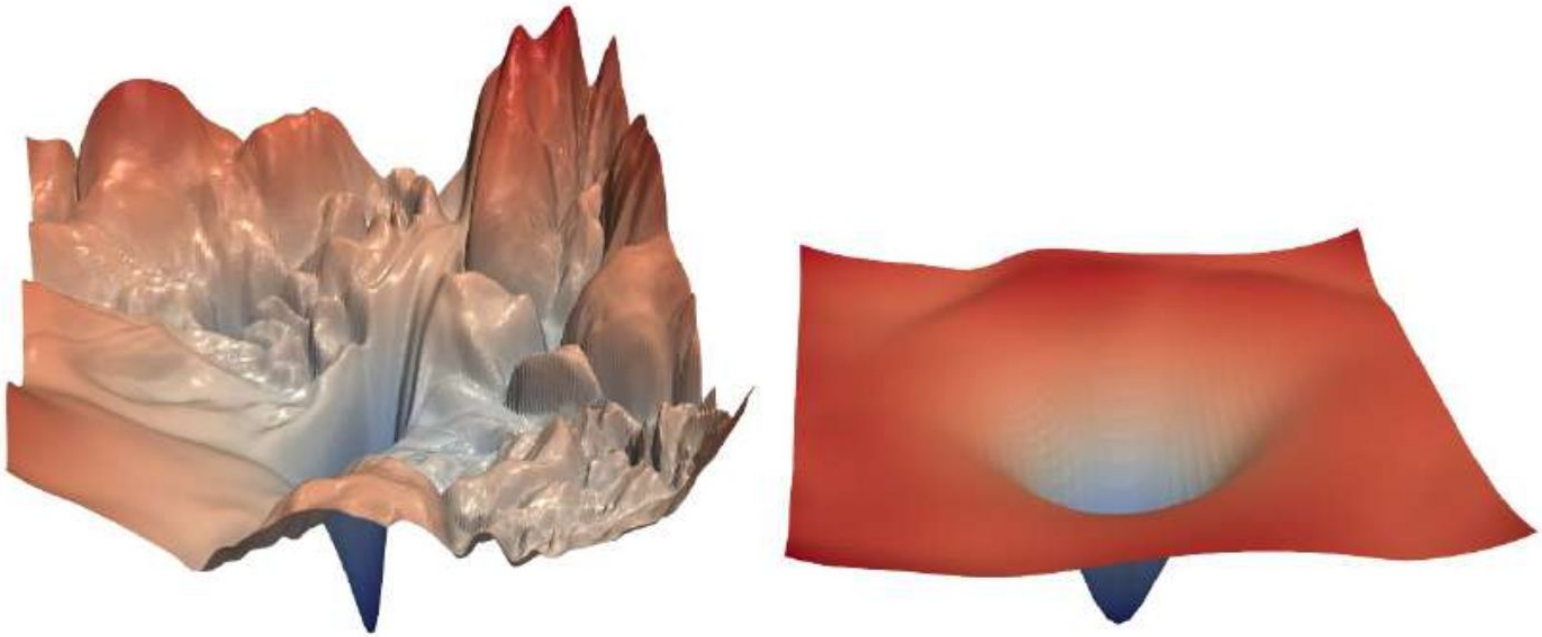


Figure 9.14 Regularization methods. The regularization methods discussed in this chapter aim to improve generalization by one of four mechanisms. Some methods aim to make the modeled function smoother. Other methods increase the effective amount of data. The third group of methods combine multiple models and hence mitigate against uncertainty in the fitting process. Finally, the fourth group of methods encourages the training process to converge to a wide minimum where small errors in the estimated parameters are less important (see also figure 20.11).

Make function smoother: Modifying the loss term



(a) (b)

Figure 9.14 (a) A visualization of the error surface for a network with 56 layers. (b) The same network with the inclusion of residual connections, showing the smoothing effect that comes from the residual connections. [From Li *et al.* (2017) with permission.]

Explicit Regularization: The Penalty Term

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^{I} \ell_i[x_i, y_i] + \lambda \cdot g[\phi] \right]$$

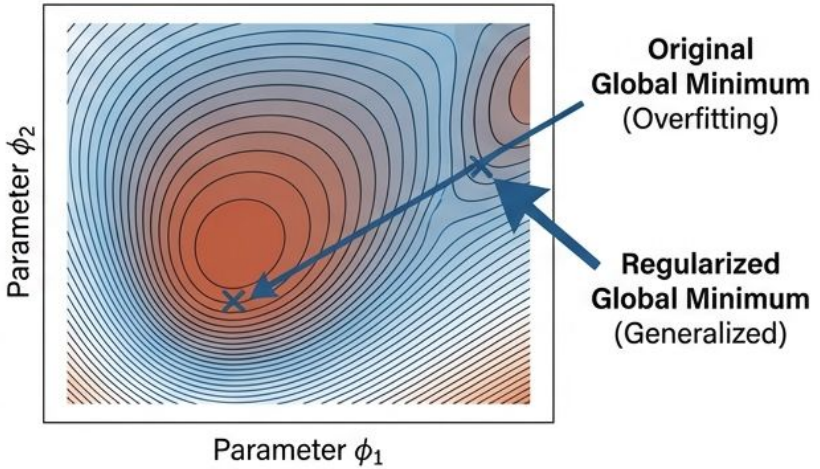
The Regularization Term
(Penalizes complexity)

The Standard Loss
(Fits the data)

The Control Knob
(Positive scalar balancing fit vs. penalty)

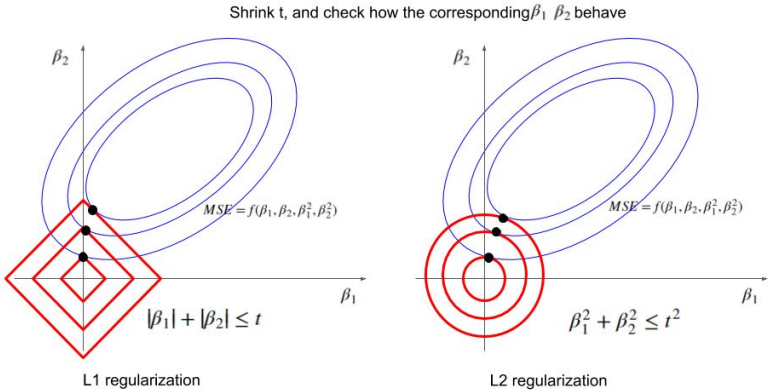
Definition: Adding an explicit term to the loss function to bias the model toward preferred parameter choices.

Probabilistic View: The regularization term acts as a prior $Pr(\phi)$ in a Maximum A Posteriori (MAP) estimation.



L1/L2 Regularization

- Fun Fact:
  - What does “L” in L1/L2 stand for?



L1/L2 Regularization

- L2 adds "***squared magnitude***" of coefficient as penalty term to the loss function.

$$Loss = Loss + \lambda \sum \beta^2$$

- L1 adds "***absolute value of magnitude***" of coefficient as penalty term to the loss function.

$$Loss = Loss + \lambda \sum |\beta|$$

- Weight Penalties -> Smaller Weights -> Simpler Model -> Less Overfit

L2 Regularization (Weight Decay)

Mechanism:

Penalizes the sum of the squares of the weights (squared L2 norm).

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^{I} \ell_i[x_i, y_i] + \lambda \sum_{j} \phi_j^2 \right]$$

JetBrins 9.5

Why it works

1. Encourages smaller weights.
2. Smaller weights → Smoother output functions (output changes less in response to input).

Equivalence

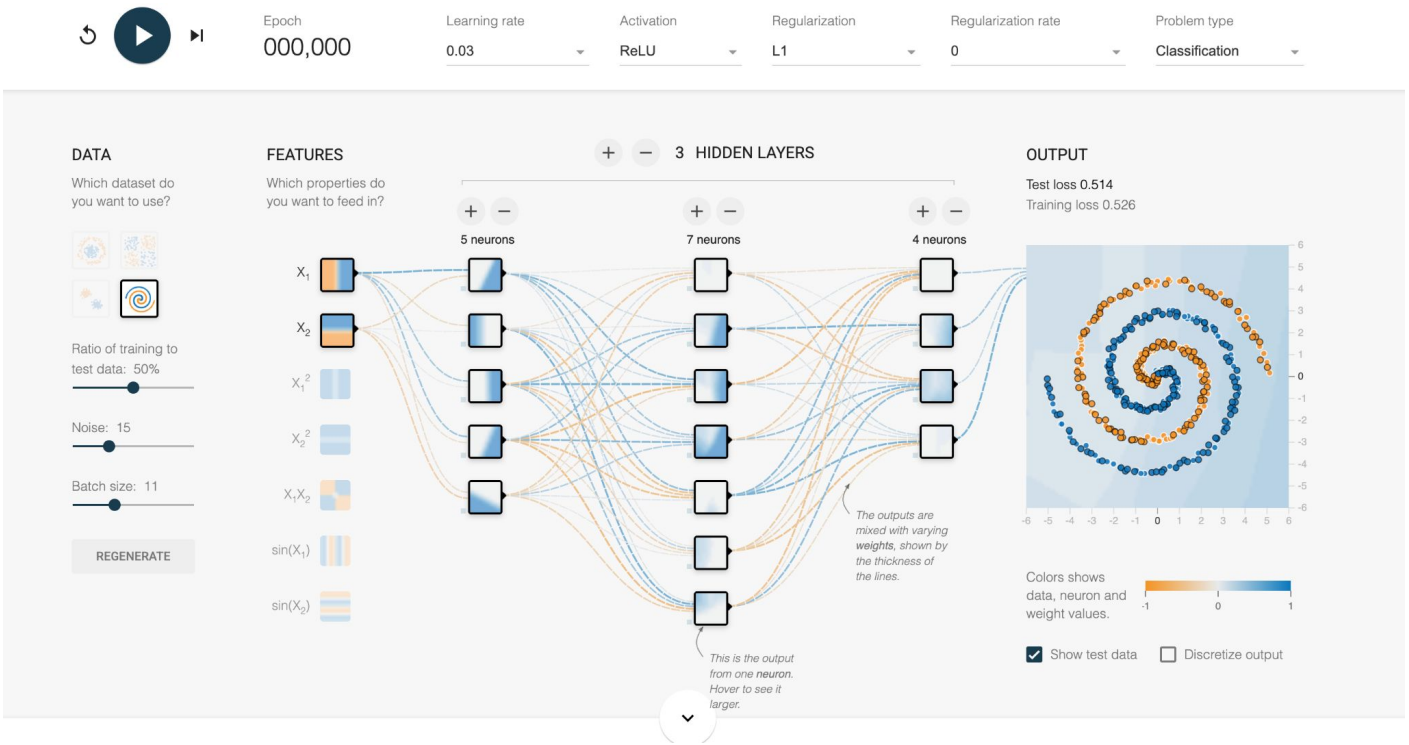
Equivalent to Ridge Regression or Tikhonov Regularization.

Note

Usually applied to weights, not biases.

L1/L2 Regularization

https://playground.tensorflow.org/



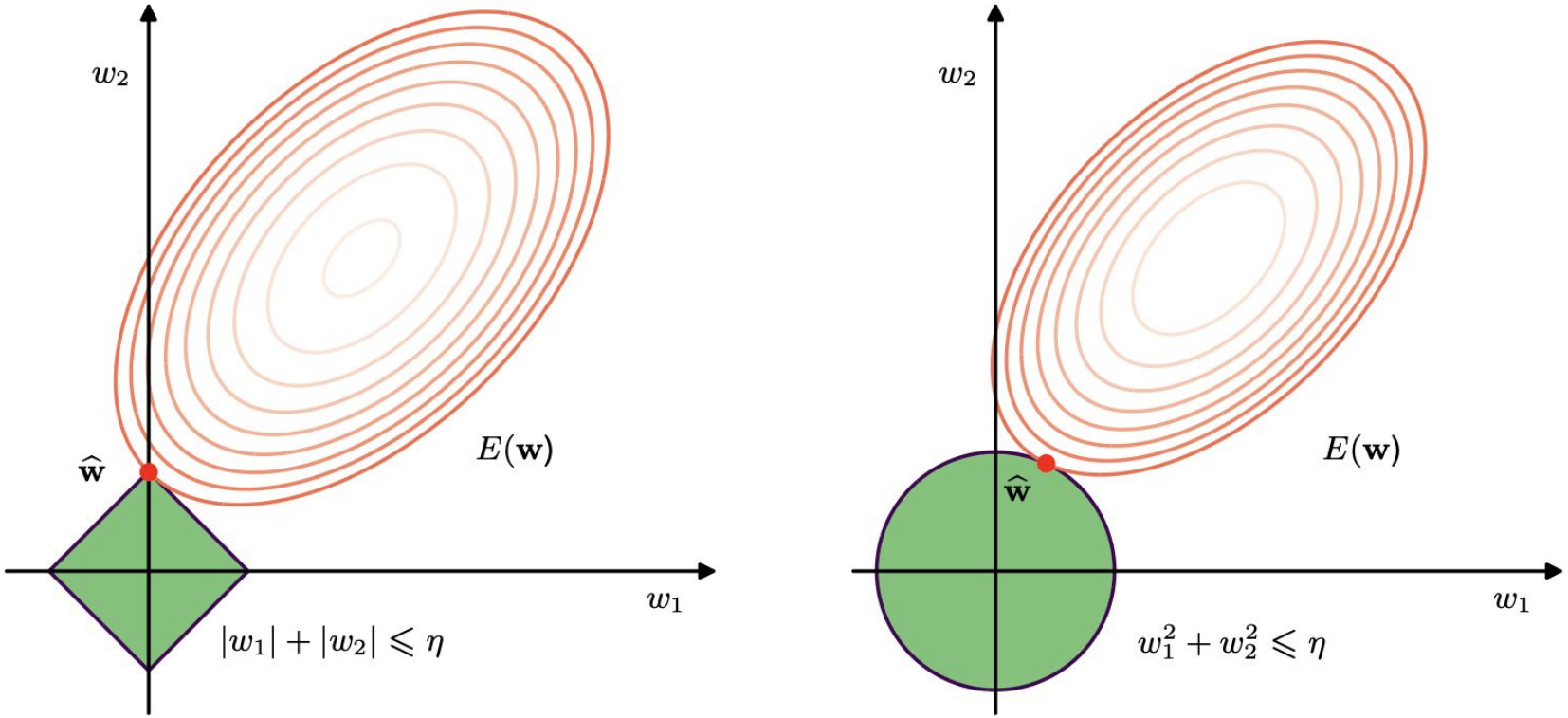


Figure 9.6 Plot of the contours of the unregularized error function (red) along with the constraint region (9.20) for the lasso regularizer $q = 1$ on the left, and the quadratic regularizer $q = 2$ on the right, in which the optimum value for the parameter vector $\mathbf{w}$ is denoted by $\widehat{\mathbf{w}}$. The lasso gives a sparse solution in which $\widehat{w}_1 = 0$, whereas the quadratic regularizer simply reduces w_1 to a smaller value.

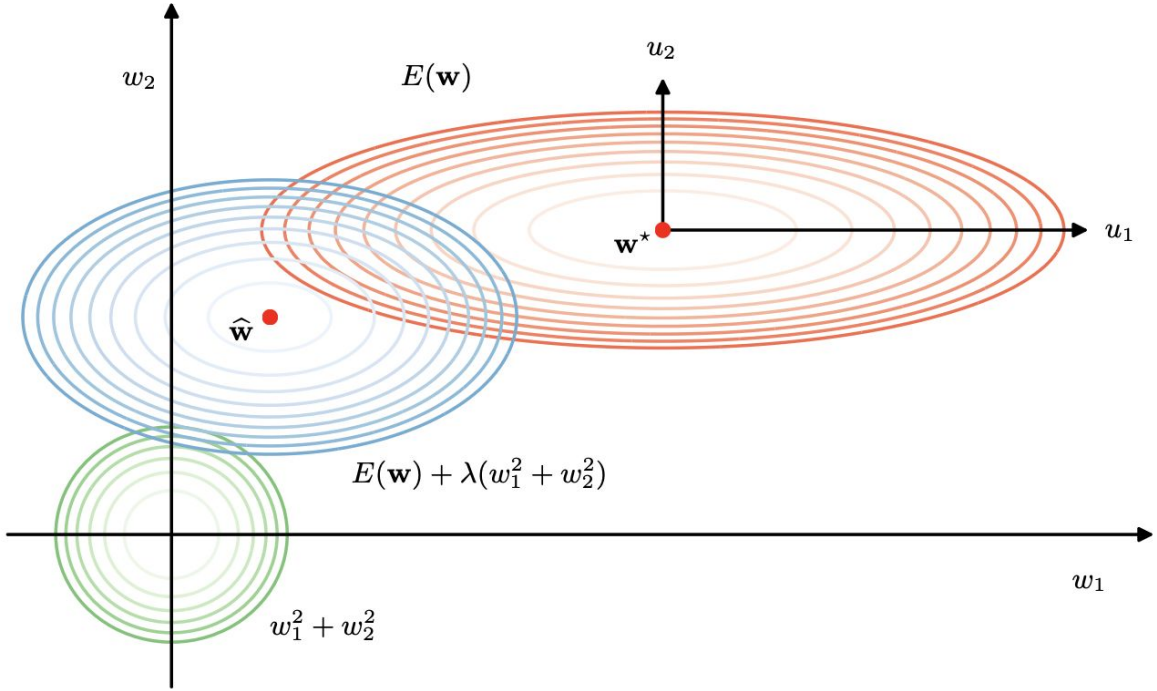
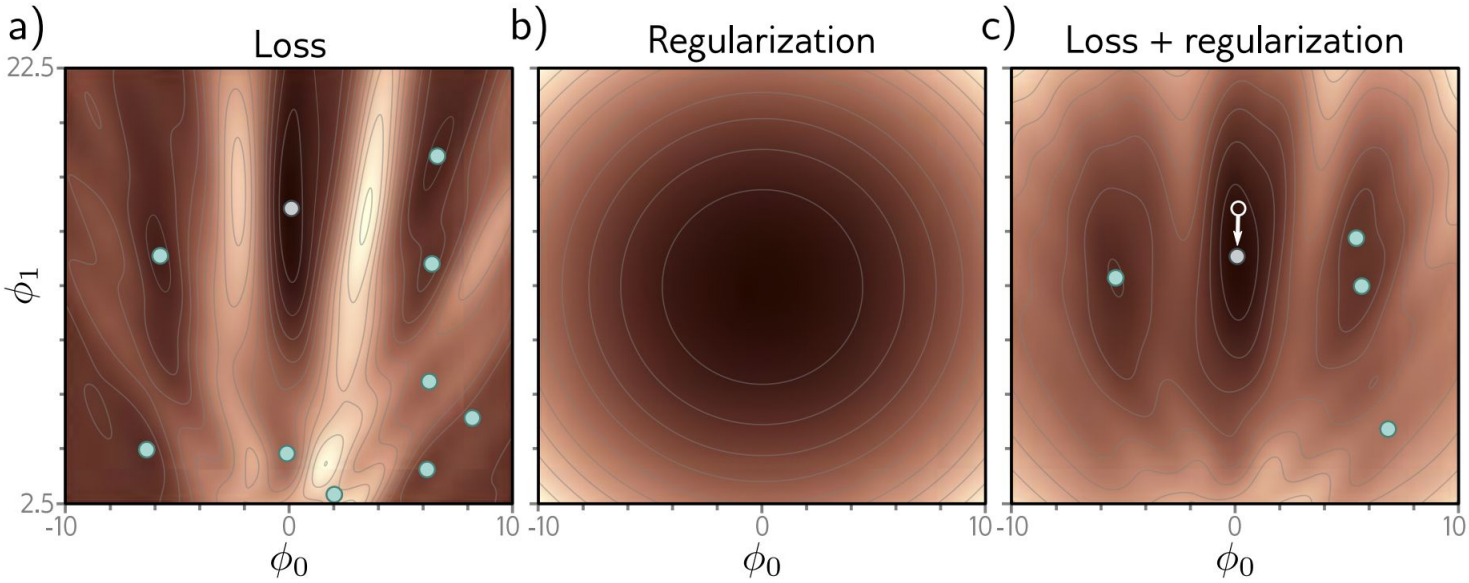


Figure 9.3 Contours of the error function (red), the regularization term (green), and a linear combination of the two (blue) for a quadratic error function and a sum-of-squares regularizer $\lambda(w_1^2 + w_2^2)$. Here the axes in parameter space have been rotated to align with the axes of the elliptical contour of the unregularized error function. For $\lambda = 0$, the minimum error is indicated by $\mathbf{w}^\star$. When $\lambda > 0$, the minimum of the regularized error function $E(\mathbf{w}) + \lambda(w_1^2 + w_2^2)$ is shifted towards the origin. This shift is greater in the direction of w_1 because the unregularized error is relatively insensitive to the parameter value, and less in direction w_2 where the error is more strongly dependent on the parameter value. The regularization term is effectively suppressing parameters that have only a small effect on the accuracy of the network predictions.

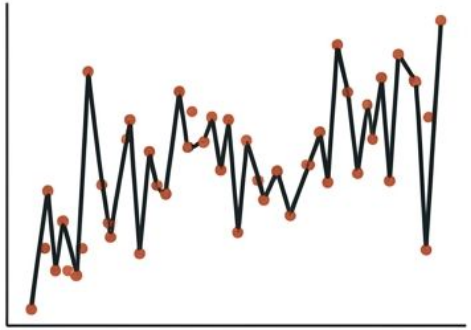


udlbook/Notebooks/Chap09/9_1_L2_Regularization.ipynb at main

Figure 9.1 Explicit regularization. a) Loss function for Gabor model (see section 6.1.2). Cyan circles represent local minima. Gray circle represents the global minimum. b) The regularization term favors parameters close to the center of the plot by adding an increasing penalty as we move away from this point. c) The final loss function is the sum of the original loss function plus the regularization term. This surface has fewer local minima, and the global minimum has moved to a different position (arrow shows change).

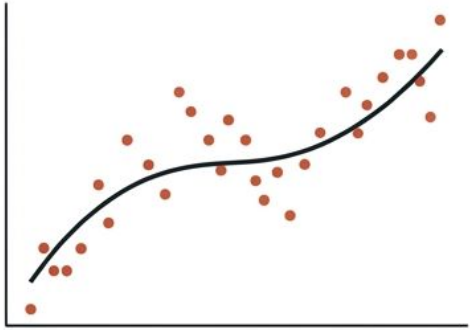
Visualizing the Impact of λ

Small λ (Under-regularized)



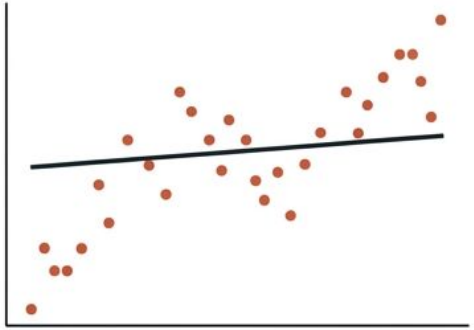
High Variance. Fits noise.

Intermediate λ (Optimal)



Good Generalization. Captures underlying structure.

Large λ (Over-regularized)



High Bias. Too simple.

Interpolation: In over-parameterized regions (no data), L2 regularization forces the function to smoothly interpolate between nearby points.

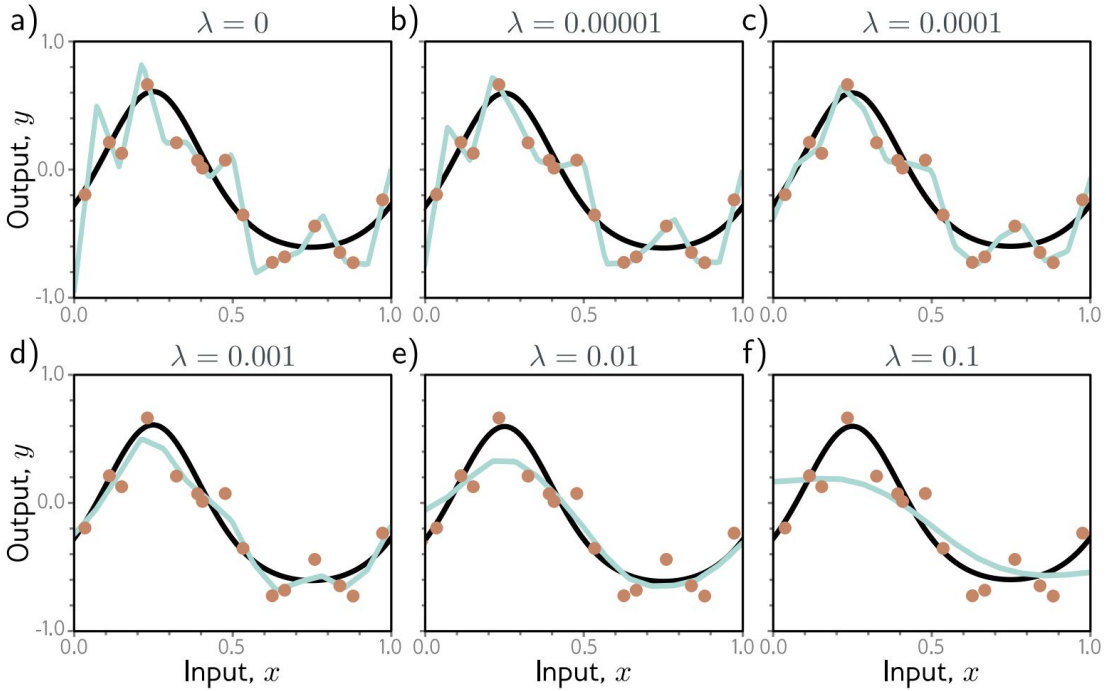


Figure 9.2 L2 regularization in simplified network with 14 hidden units (see figure 8.4). a–f) Fitted functions as we increase the regularization coefficient λ. The black curve is the true function, the orange circles are the noisy training data, and the cyan curve is the fitted model. For small λ (panels a–b), the fitted function passes exactly through the data points. For intermediate λ (panels c–d), the function is smoother and more similar to the ground truth. For large λ (panels e–f), the regularization term overpowers the likelihood term, so the fitted function is too smooth and the overall fit is worse.

L1/L2 Regularization

- Regularization works on assumption that smaller weights generate simpler model and thus helps avoid overfitting.

- Why?

    - Robustness and Sparsity

Robustness (Against Outliers)

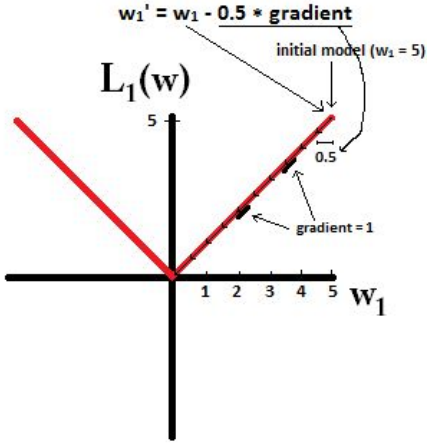
- L1>L2
- The loss of outliers increase
  - Exponentially in L2
  - Linearly in L1
- L2 pays more efforts to deal with outliers -> Less Robust

Sparsity

- L1>L2
- L1 zeros out coefficients, which leads to a sparse model
- L1 can be used for feature (coefficients) selection
  - Unimportant ones have zero coefficients
- L2 will produce small values for almost all coefficients

- E.g: When applying L1/L2 to a layer with 4 weights, the results might look like
  - L1: 0.8, 0, 1, 0
  - L2: 0.3,0.1,0.3, 0.2

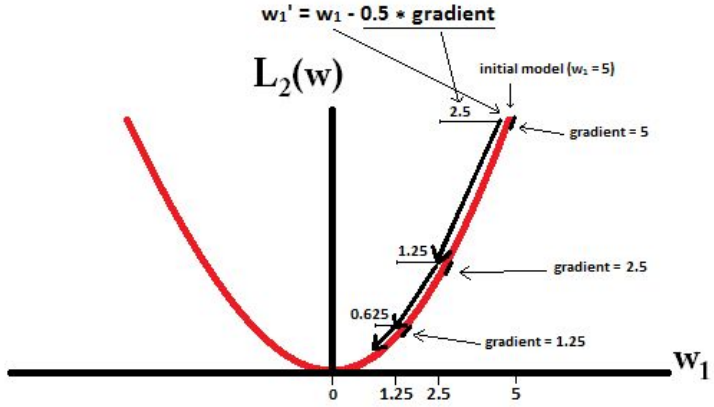
Sparsity



$$w_1 = w_1 - 0.5 * gradient$$

gradient is constant (1 or -1)

w1: 5->0 in 10 steps



$$w_1 = w_1 - 0.5 * gradient$$

gradient is smaller over time (w2: 5->0 in a big number of steps

Problem 9.2: The Regularized Gradient

Problem

Question: How do the gradients of the loss function change when L2 regularization is added?

Standard Loss Gradient:

$$\nabla L_{\text{original}} = \frac{\partial L}{\partial \phi}$$

L2 Penalty:

$$R(\phi) = \lambda \sum \phi_j^2$$

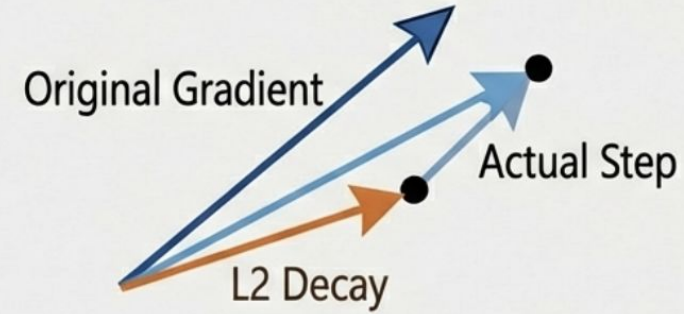
Solution & Visual

The Derivative:

$$\frac{\partial}{\partial \phi}(\lambda \phi^2) = 2\lambda\phi$$

New Update Rule:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \left(\nabla L_{\text{original}} + 2\lambda\phi_t \right)$$



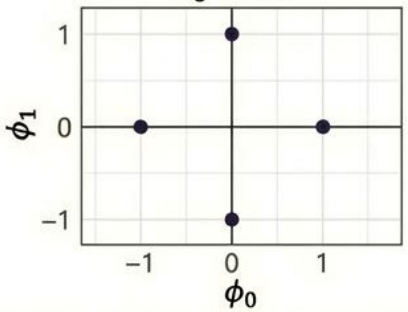
Problem 9.6: Visualizing Norms

- **Problem Statement:** Consider a model with parameters $\phi = [\phi_0, \phi_1]^T$. Draw the contours of the regularization terms for different p-norms.
- **General Formula:** L_p norm $= \sum |\phi_d|^p$

- **The Shapes to Draw:**
- L_0 (Count of non-zeroes)
- $L_{0.5}$ (Hyper-concave)
- L_1 (Lasso / Diamond)
- L_2 (Ridge / Circle)

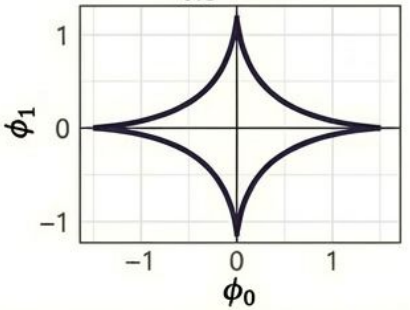
The Geometry of Constraints

L_0 Norm



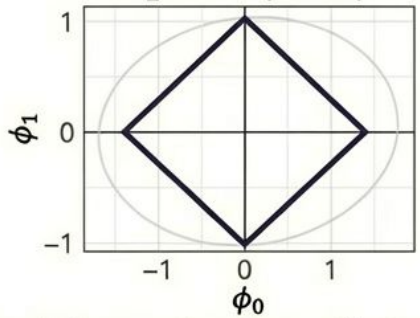
Sparse (Penalty 0 on axes, ∞ elsewhere)

$L_{0.5}$ Norm



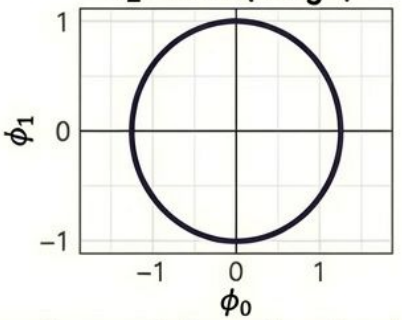
Hyper-concave (Aggressive shrinkage)

L_1 Norm (Lasso)



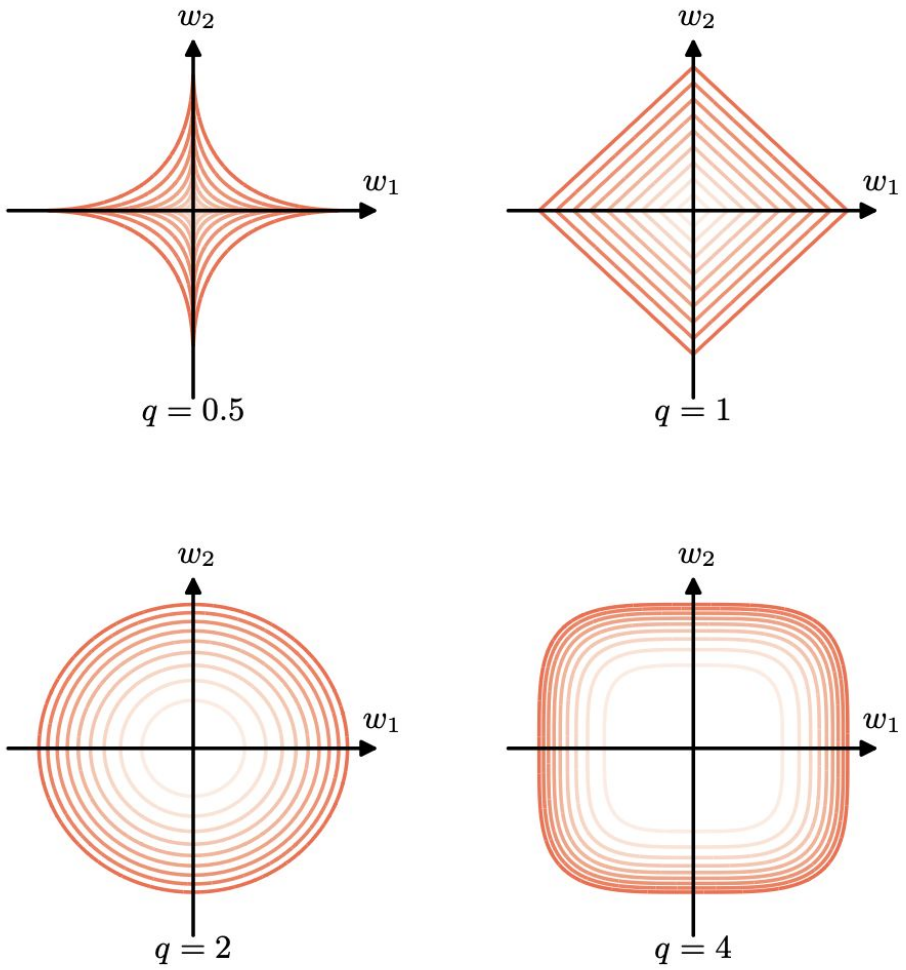
Diamond. Corners touch axes first → Sparsity.

L_2 Norm (Ridge)



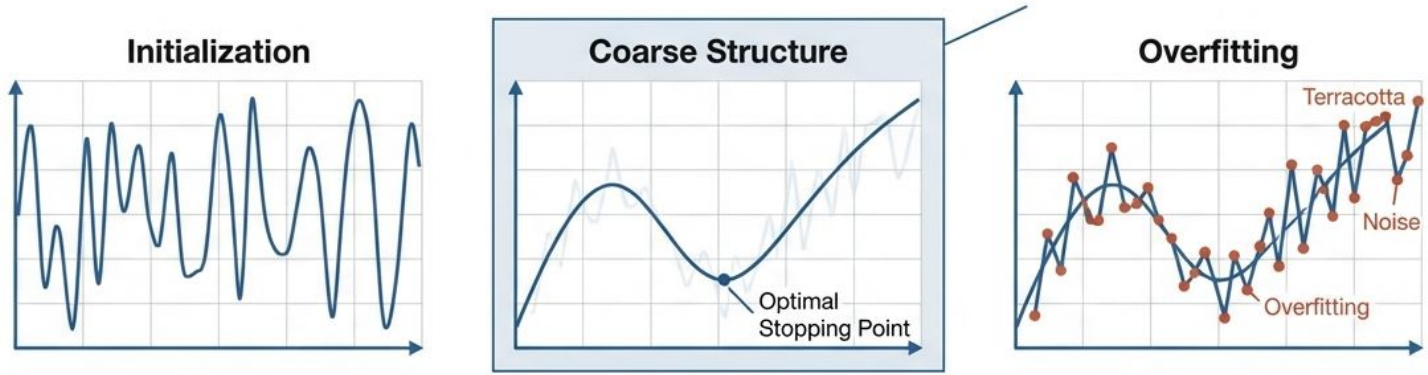
Circle. Touches tangentially → Small weights, but not zero

Figure 9.5 Contours of the regularization term in (9.18) for various values of the parameter q.



Heuristics: Rules of thumb

Heuristics I: Early Stopping



Concept: Stop training before the model converges on the training noise.

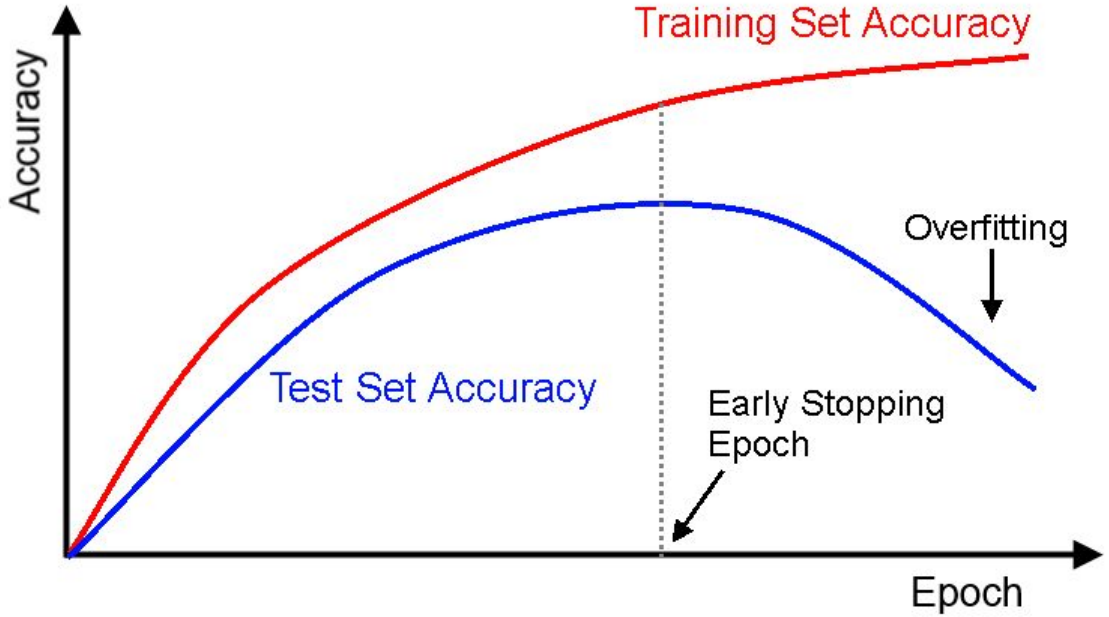
The Process
- **Initialize** weights to small values.
- Monitor validation set performance.
- Save parameters when validation error is lowest.

Connection to L2: Since weights start small, stopping early prevents them from growing large (similar to `Weight Decay`).

Early Stopping

- There is point during training a large neural net when the model will stop generalizing (performing well on test/validation sets) and only focus on learning the statistical noise in the training dataset.

- Solution
  - Stop whenever generalization errors increases



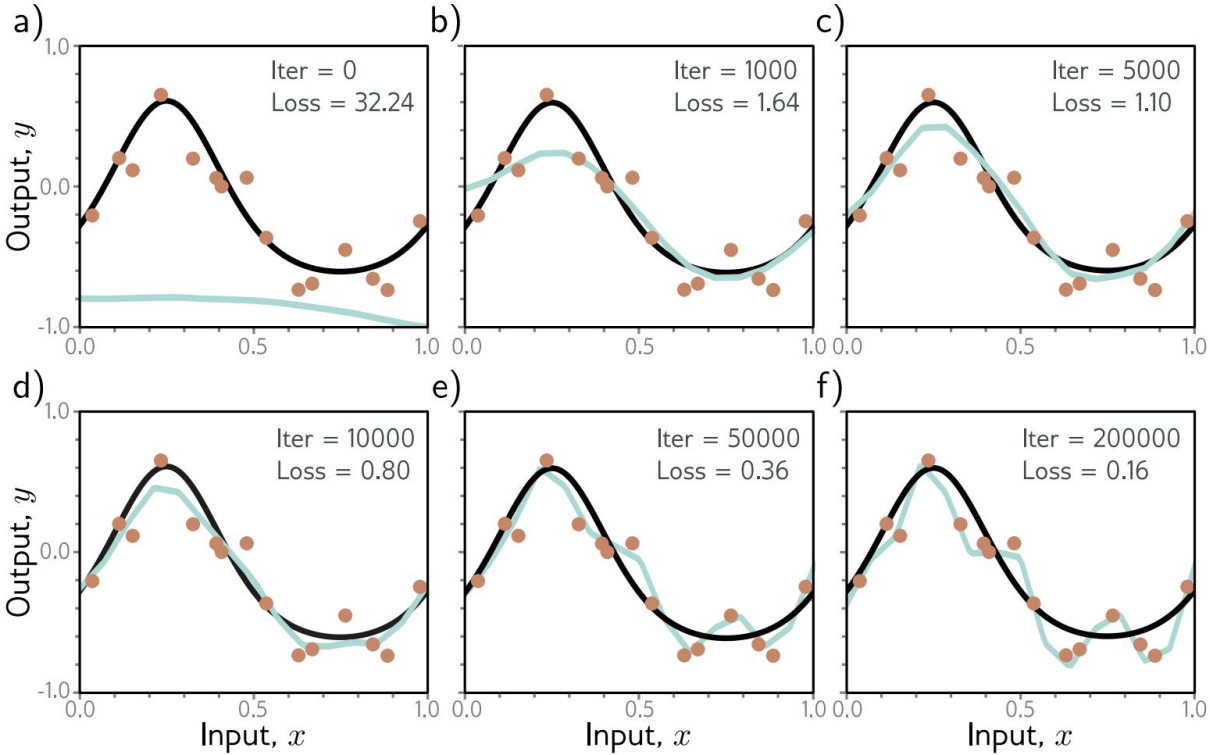


Figure 9.6 Early stopping. a) Simplified shallow network model with 14 linear regions (figure 8.4) is initialized randomly (cyan curve) and trained with SGD using a batch size of five and a learning rate of 0.05. b–d) As training proceeds, the function first captures the coarse structure of the true function (black curve) before e–f) overfitting to the noisy training data (orange points). Although the training loss continues to decrease throughout this process, the learned models in panels (c) and (d) are closest to the true underlying function. They will generalize better on average to test data than those in panels (e) or (f).

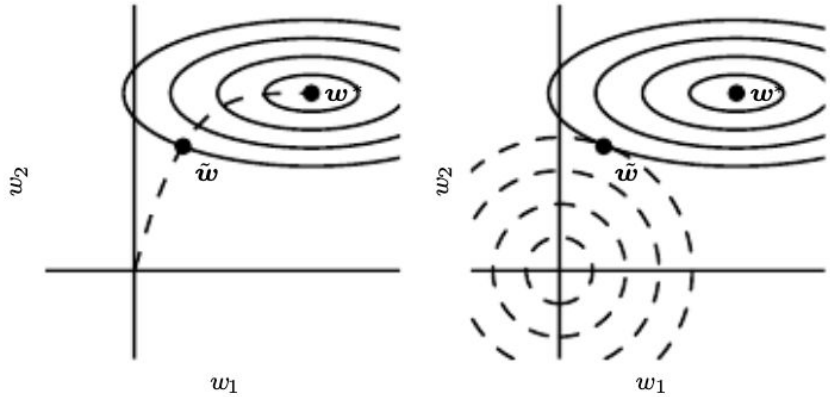


Figure 7.4: An illustration of the effect of early stopping. *(Left)*The solid contour lines indicate the contours of the negative log-likelihood. The dashed line indicates the trajectory taken by SGD beginning from the origin. Rather than stopping at the point $\boldsymbol{w}^*$ that minimizes the cost, early stopping results in the trajectory stopping at an earlier point $\tilde{\boldsymbol{w}}$. *(Right)*An illustration of the effect of L^2 regularization for comparison. The dashed circles indicate the contours of the L^2 penalty, which causes the minimum of the total cost to lie nearer the origin than the minimum of the unregularized cost.

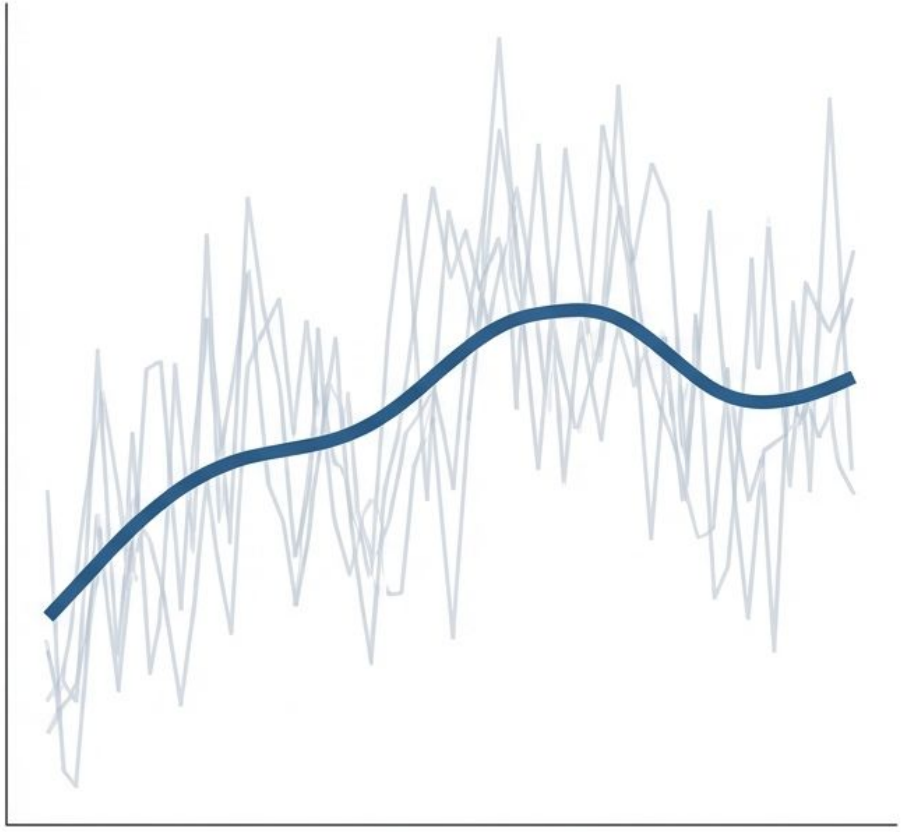
Heuristics II: Ensembling

Definition:
Train multiple models and average their predictions.

Why it works:
Assumption that model errors are independent. Averaging cancels out the noise.

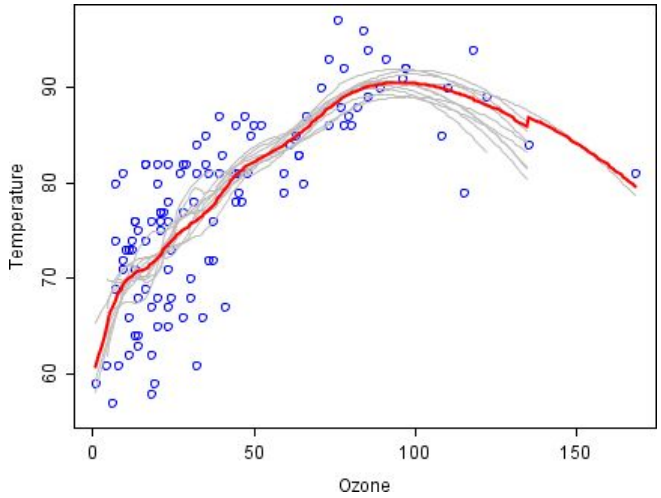
Methods:
- Random Initialization: Different starting points find different solutions.
- Bagging (Bootstrap Aggregating): Re-sampling training data with replacement.

Trade-off:
Improves performance reliability at the cost of increased training/inference compute.



Ensemble Models

- Averaging multiple models to create a final model with low variance



Ensemble Models - Bagging

- How it works
  - Train multiple models on different subsets of data
  - Combine those models into a final model

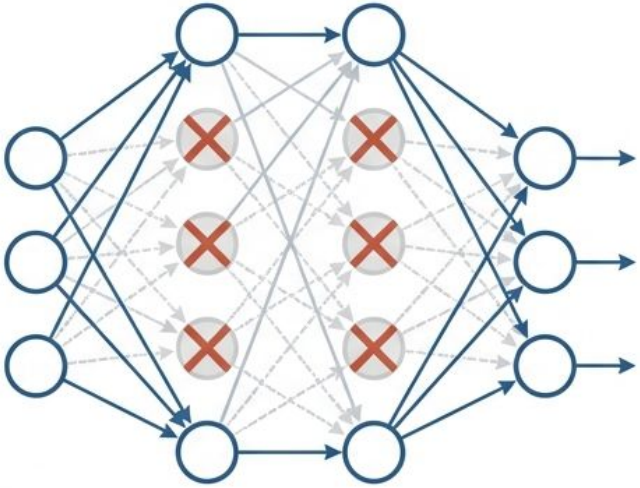
- Characteristics
  - Each sub-model is trained separately
  - Each sub-model is normally overfit
  - The combination of those overfit models produce a less overfit model overall

https://github.com/udlbook/udlbook/blob/main/Notebooks/Chap09/9_3_Ensembling.ipynb

Heuristics III: Dropout

Mechanism:

Randomly clamp a subset (e.g., 50%) of hidden units to zero during each training iteration.



Effect:

Removes sharp changes in the function that don't contribute to the overall loss.

The "Conspiracy" Analogy:

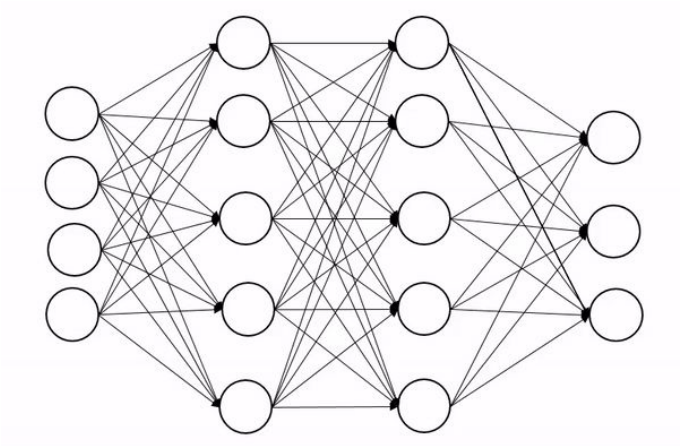
Prevents neurons from relying on specific "kinks" or corrections from other specific neurons. Forces the network to be redundant and robust.

Inference Rule:

At test time, all units are active. Weights must be scaled by $(1 - p)$.

Dropout

- How it works
  - Randomly selected neurons are ignored during each training step.
  - Dropped neurons don’t have effect on next layers.
  - Dropped neurons are **not updated** in backward training.



- Questions:
  - What’re the ideas?
  - Why dropout help to reduce overfit?

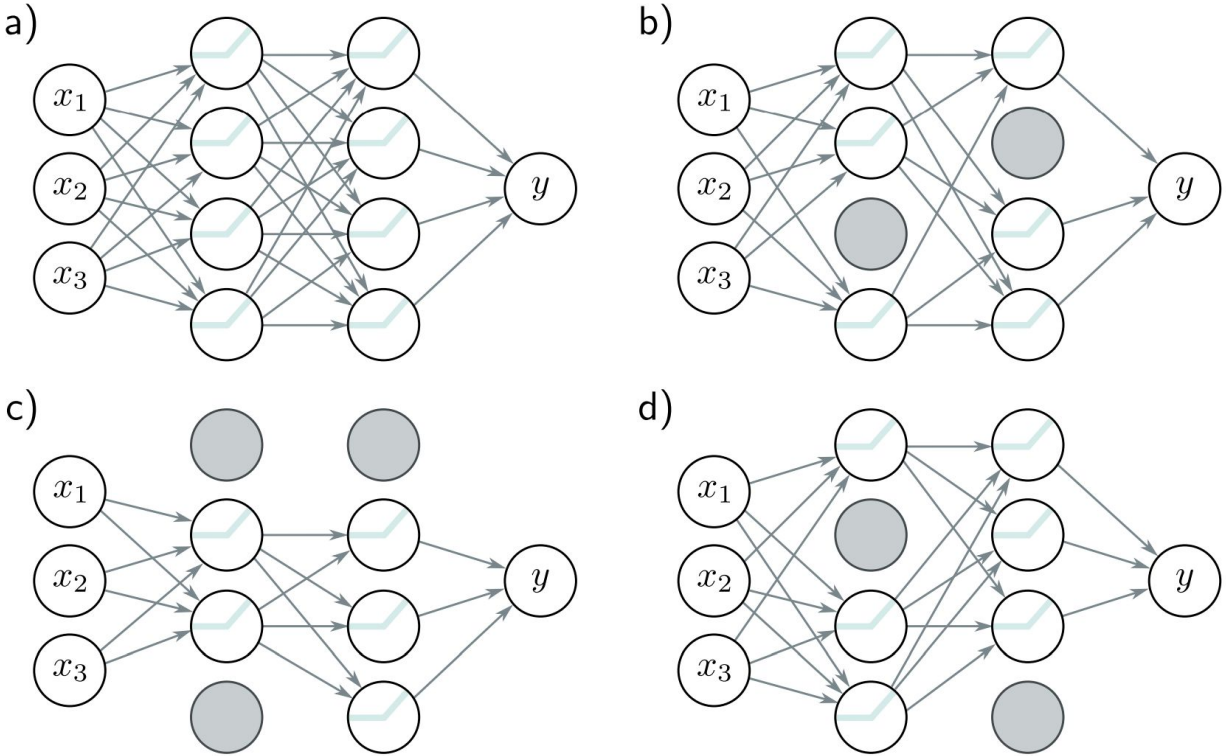


Figure 9.8 Dropout. a) Original network. b–d) At each training iteration, a random subset of hidden units is clamped to zero (gray nodes). The result is that the incoming and outgoing weights from these units have no effect, so we are training with a slightly different network each time.

Dropout

- Removing units from base model effectively creates a subnetwork.
- All those subnetworks are trained implicitly together with all **parameters shared** (different from bagging)
- At predict mode, all learned units are activated, which averages all trained subnetworks

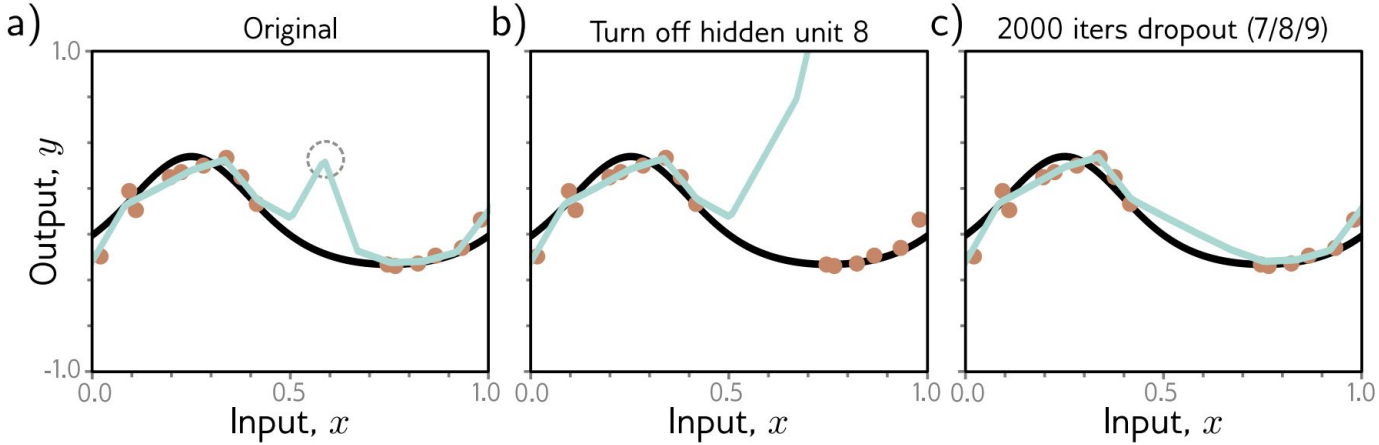


Figure 9.9 Dropout mechanism. a) An undesirable kink in the curve is caused by a sequential increase in the slope, decrease in the slope (at circled joint), and then another increase to return the curve to its original trajectory. Here we are using full-batch gradient descent, and the model (from figure 8.4) fits the data as well as possible, so further training won't remove the kink. b) Consider what happens if we remove the eighth hidden unit that produced the circled joint in panel (a), as might happen using dropout. Without the decrease in the slope, the right-hand side of the function takes an upwards trajectory, and a subsequent gradient descent step will aim to compensate for this change. c) Curve after 2000 iterations of (i) randomly removing one of the three hidden units that cause the kink and (ii) performing a gradient descent step. The kink does not affect the loss but is nonetheless removed by this approximation of the dropout mechanism.

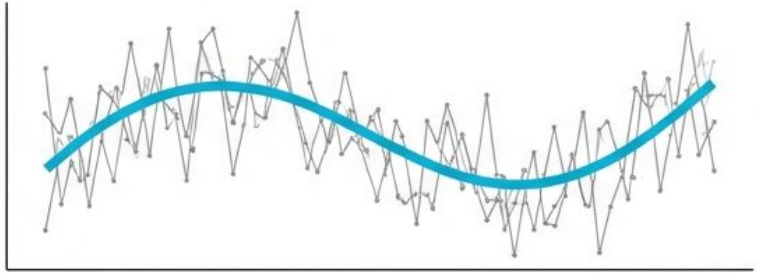
Dropout - Ensemble Models for NN

- Can we apply Bagging for Neural Network?
  - It’s computationally prohibitive

- Dropout aims to solve this problem by providing a method to combine multiple models with practical computation cost.

Heuristics IV: Noise & Label Smoothing

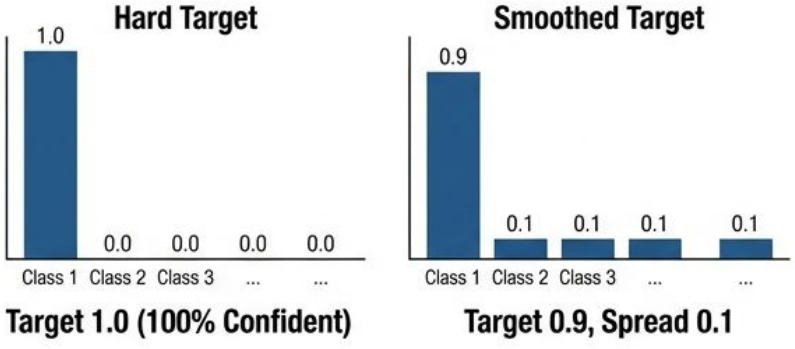
Input & Weight Noise



Input Noise forces the model to be insensitive to small perturbations (Invariance).

Weight Noise encourages convergence to wide, flat minima.

Label Smoothing



Problem: Maximum likelihood forces the model to be 100% confident (target 0 or 1), leading to overfitting.

Solution: Target a probability of e.g., 0.9 for the correct class and 0.1 spread among others.

Result: Prevents the model from pushing weights to infinity to achieve certainty.

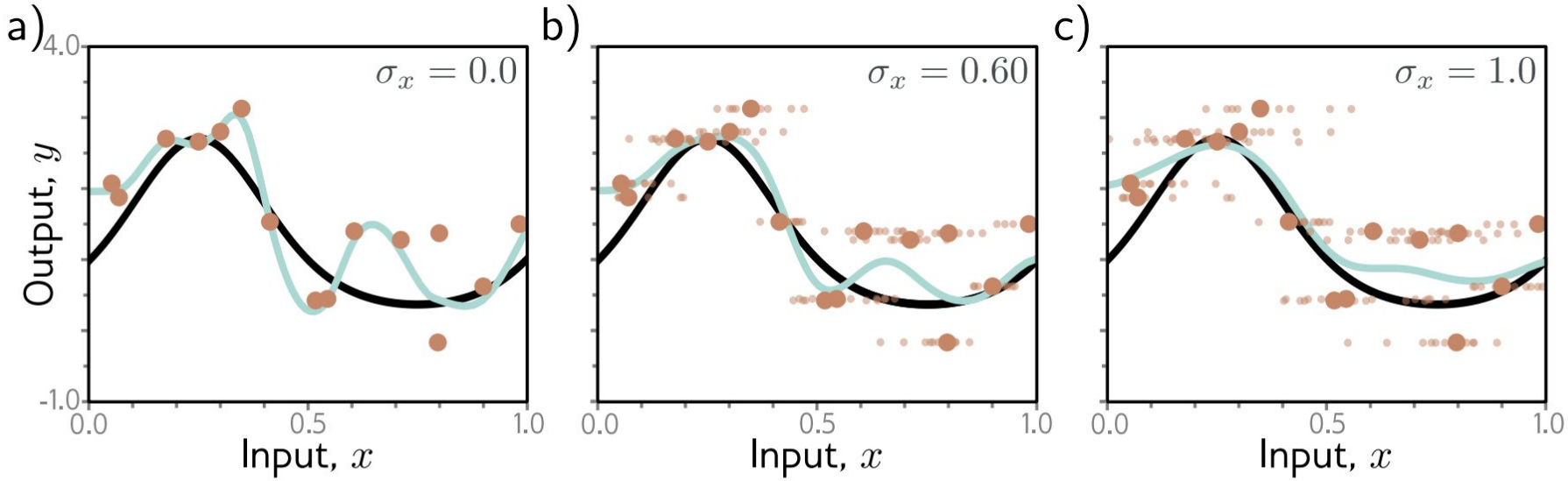


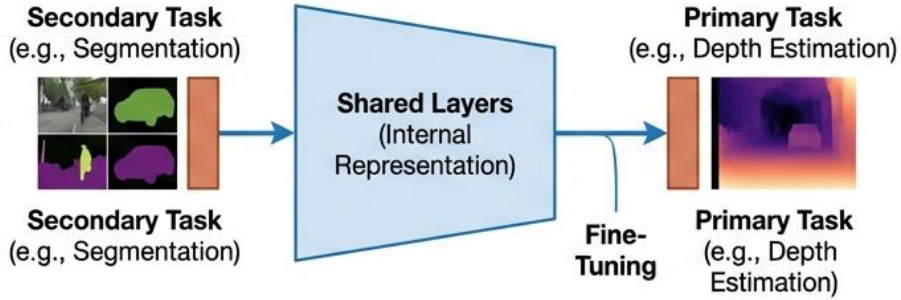
Figure 9.10 Adding noise to inputs. At each step of SGD, random noise with variance σ_x^2 is added to the batch data. a–c) Fitted model with different noise levels (small dots represent ten samples). Adding more noise smooths out the fitted function (cyan line).

Increase the data

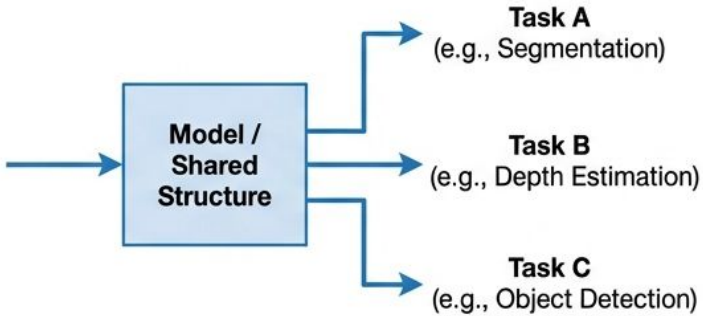
Transfer & Multi-Task Learning

Transfer Learning:

1. Train on a data-rich secondary task (e.g., ImageNet).
2. “Transfer” the internal representation (weights) to the target task.
3. Fine-tune the final layers.



Multi-Task Learning:
Train a single model to solve multiple problems concurrently.



Benefit: The model learns a robust internal structure that captures the physics of the world, not just the peculiarities of a small dataset.

Data Augmentation

Original

Red color casting

Rotation

Distortion

Concept:
Expand the dataset by transforming examples in ways that preserve the label.

Invariance:
Teaching the model that a **rotated bird bird is still a bird.**

Techniques:
- Images: Rotation, flipping, cropping, color shifting.
- Audio: Pitch shifting, noise injection.
- Text: Synonym substitution, back-translation.

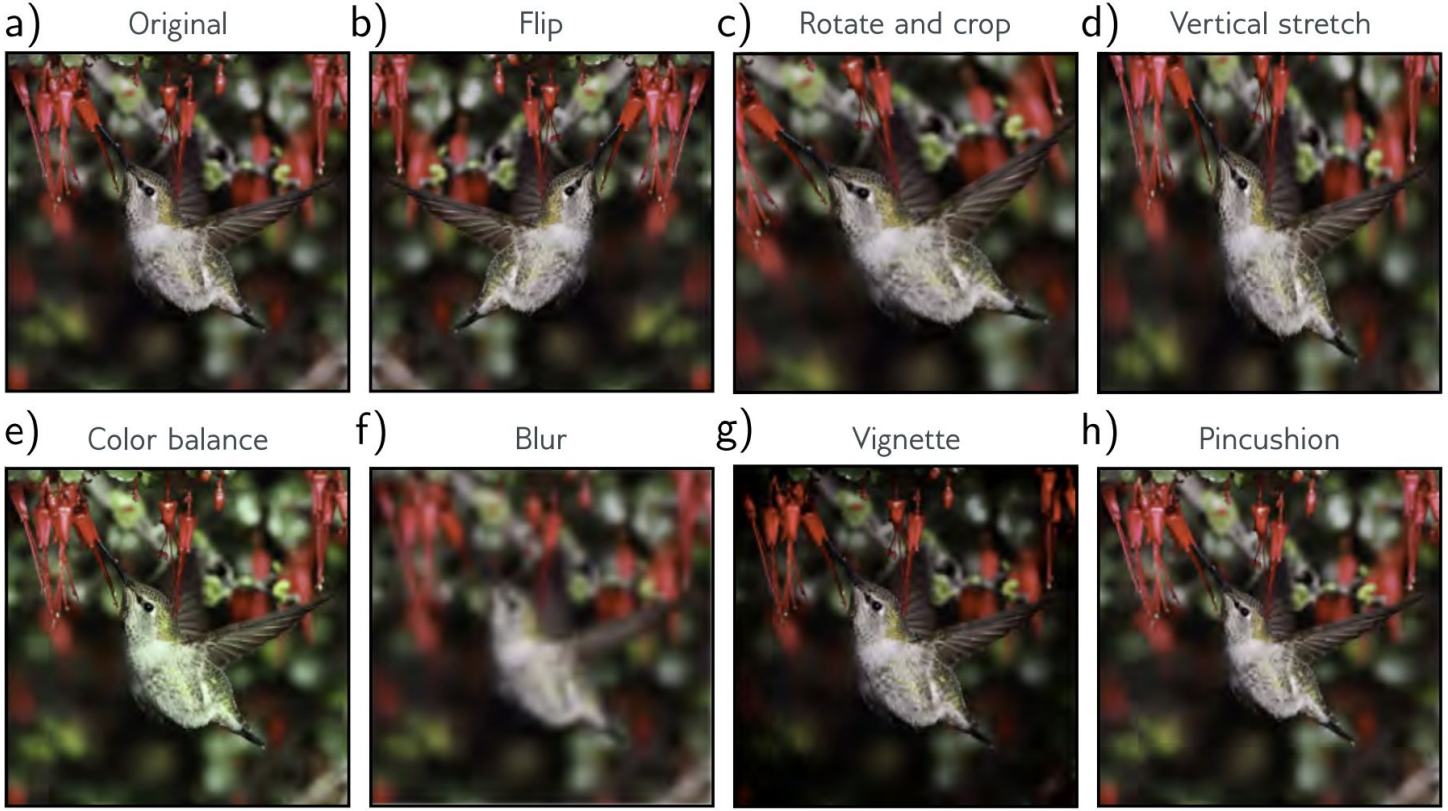


Figure 9.13 Data augmentation. For some problems, each data example can be transformed to augment the dataset. a) Original image. b–h) Various geometric and photometric transformations of this image. For image classification, all these images still have the same label, "bird." Adapted from Wu et al. (2015a).

Conclusion

“Regularization is not just a tool; it is a mindset.”

We move from asking ‘How well does this fit the data we have?′ to ‘How well does this describe the world?′

Final Thought: The best model is often not the one with 0% training error, but the one that sacrifices some precision on the training set to gain robustness on the test set.

Summary: The Four Principles of Regularization

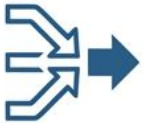
Make it Smoother

- L2 Regularization
- Early Stopping

More Data

- Data Augmentation
- Transfer Learning

Combine Models



- Ensembling
- Dropout

Wider Minima

- Weight Noise
- SGD Implicit Regularization

Newton School of Technology

Thank you all for a great session please fill the feedback form.